

Royal Education Society's

College of Computer Science and Information Technology, Latur.

Affiliated to

Swami Ramanand Teerth Marathwada University, Nanded.

A Project Report

on

VRITTACARE: Student Mental Health Score Prediction System

Submitted for the award of degree of

Bachelor of Science (Data Science)

By:

Vaibhav Badrinarayan Solanke

Roll No: A37 Exam Seat No: _____

Rutuja Satish Dhumal

Roll No: A59 Exam Seat No: _____

In

Academic Year: 2026 – 2027

Royal Education Society's

COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR.

CERTIFICATE

This is to certify that the project entitled **"VRITTACARE: Student Mental Health Score Prediction System"** has been carried out by **Vaibhav Badrinarayan Solanke** and **Rutuja Satish Dhumal** under my guidance in partial fulfilment of the degree i.e. **Bachelor of Science in Data Science** of SRTMU, Nanded during the academic year 2026-2027.

Date: 22 October 2026 Place: Latur, Maharashtra

Mr. S. T. Somwanshi
Guide

Mr. N. D. Jagtap
Head of Department

ACKNOWLEDGEMENT

We would like to convey our gratitude to Dr. R. S. Awasthi, Principal of College of Computer Science and Information Technology, Latur who gave us necessary information and guidance for project.

We are grateful to Swami Ramanand Teerth Marathwada University, Nanded for giving an opportunity to deliver project.

We would like to thank Project In-Charge Mr. S. Shaikh and Project Guide Mr. S. T. Somwanshi who guided us through doing these project development process, provided with invaluable advice, helped us in difficult periods and provided practical assistant for our project. Their willingness to motivate us contributed tremendously to the success of this project.

We would like to express our special thanks of gratitude to our Head of the Department of Data Science, Mr. N. D. Jagtap who helped us a lot in finalizing this project.

Besides we would like to thank all staff members who helped us by giving advice and providing equipment which we needed.

Last but not in least we would like to thank all who helped and motivated us.

With Sincere Thanks,

1. Vaibhav Badrinarayan Solanke
2. Rutuja Satish Dhumal

Index

Sr. No.	Topic Name	Page No.
1	Abstract	1
2	Introduction:	
	2.1 Project Overview	2
	2.2 Project Module	3
	2.3 Project Plan (Gantt chart)	3
3	Project Requirement:	
	3.1 Hardware Requirement	5
	3.2 Software Requirement	5
	3.3 Field Work	6
	3.4 Functional and Non-Functional Requirements	9
4	System Design:	
	4.1 Database Design (Tables Schema)	12
	4.2 E-R Diagram	13
	4.3 Data Flow Diagram (First Level)	14
5	Designing	17
6	Coding	30
7	Future scope of project	49
8	Conclusion	50
9	Bibliography	51
	9.1 Books	51
	9.2 Websites	51

1. ABSTRACT

Mental well-being is an important part of a student's academic life. Everyday habits such as sleep, study time, physical activity, time spent on the phone and social media, and stress are closely connected with how a student feels, but students rarely get a simple and structured way to look at their own routine. VRITTACARE is a student-focused web application that uses machine learning to turn these everyday habits into an understandable wellness score, clear observations and small, practical next steps.

A student creates an account and completes a short-guided assessment of twelve inputs: age, gender, country, academic level, most-used social media platform, purpose of use, average daily usage hours, daily phone unlocks, study hours, physical activity hours, sleep hours per night and stress level. The inputs are validated and sent to a Python FastAPI service, which applies a saved preprocessing pipeline and a trained Random Forest regression model to predict a Mental Health Score on a 10-point scale. The result is shown as a score gauge, a project-defined category (Low, Moderate, Good or Excellent / Stable), feature-level observations grouped as Needs Attention, Can Improve / Watch and Favorable / Stable, personalised tips, a 3D wellness profile and an AI-based Wellness Assistant.

The model was developed from the dataset "Student Social Media And Mental Health Impact", which has 5,000 records and 13 columns (4,998 records after removing two duplicates). After one-hot encoding, ordinal encoding and scaling, 76 input features were used with an 80:20 train-test split. Four regression algorithms – Linear Regression, Decision Tree, Random Forest and Gradient Boosting – were compared, and Random Forest performed best. After hyperparameter tuning with randomized search and five-fold cross-validation, the final model achieved a Mean Absolute Error of 0.2670, a Root Mean Squared Error of 0.3586 and an R^2 score of 0.9280 on 1,000 unseen test records.

The frontend is built with React, TypeScript, TanStack Start and Tailwind CSS, the prediction service is built with FastAPI, and both are deployed on Render. Supabase provides authentication and user-scoped storage of assessments, prediction results and chat history, and the Wellness Assistant uses the Google Gemini API. VRITTACARE is an educational tool for self-reflection. It does not diagnose any mental-health condition and is not a substitute for a qualified professional.

Keywords: VRITTACARE, Student Mental Health, Score Prediction, Machine Learning, Random Forest Regressor, scikit-learn, FastAPI, React, TypeScript, Supabase, Google Gemini, Wellness Assistant, Render.

2. INTRODUCTION

2.1 Project Overview

Background and Problem Statement

College life brings academic pressure, irregular sleep, long hours on smartphones and little time for physical activity. These habits are closely connected with how students feel, yet most students have no simple way to look at their own routine. Existing resources are mostly clinical questionnaires or general articles. There is a need for a private, non-judgemental tool in which a student can enter daily habits, receive an estimated wellness score learned from data, and get practical suggestions for small improvements, without the result being presented as a medical diagnosis.

About the Project

VRITTACARE is an end-to-end student mental-health score prediction and wellness-insight system that combines a trained regression model with a calm, interactive web application. The journey is simple: Assessment → Prediction → Understanding → Small, practical actions.

Objectives

1. To study how social media use, lifestyle habits and stress relate to the Mental Health Score of students.
2. To clean, explore and prepare the dataset and build a preprocessing pipeline that is also used at prediction time.
3. To compare regression algorithms and to select, tune and save the best-performing model.
4. To expose the model through a validated FastAPI REST API.
5. To build a web application with sign-in, assessment, results, insights, a 3D wellness profile and a Wellness Assistant.
6. To store each student's data privately in Supabase and to present every result as self-reflection, not diagnosis.

Scope and Limitations

The system covers data analysis, model development, a prediction API, a deployed web application and an assistant for explanations. The Mental Health Score is a model-generated estimate based on patterns in the training dataset. Category labels and feature rules are project-defined guidance and not clinical thresholds, and the application must not be used for clinical decisions or emergency support.

2.2 Project Module

The project is divided into the following modules. Each module has a clear responsibility, which kept the development work organised between the team members.

Table 2.1: Project modules

Sr.	Module	Description
1	Authentication	Account creation and sign-in using email and password or Google through Supabase Auth. Protected pages are available only to signed-in students, and a profile with a display name is created for every user.
2	Dashboard	Welcome screen with the latest score gauge, category, wellness profile bars (sleep, study, activity, screen usage, stress) and small next-step suggestions.
3	Assessment	Guided form with twelve inputs. Every value is validated against allowed ranges before it is submitted.
4	Prediction	A server function sends the assessment to the FastAPI /predict endpoint. The backend preprocesses the inputs, predicts the score with the Random Forest model and returns the score, category and feature analysis, which are then saved.
5	Results	Shows the score gauge, the category, the wellness profile with a status for every lifestyle signal and the most useful area to focus on.
6	Insights and Tips	Groups observations into Needs Attention, Can Improve / Watch and Favorable / Stable and suggests a short focus path of small actions.
7	3D Wellness Profile	Interactive 3D view of the five lifestyle signals of the latest assessment.
8	Wellness Assistant	Chat panel powered by the Google Gemini API. It uses the student's latest assessment, result and recent chat history and answers in a supportive, non-diagnostic way.
9	About Project	Explains the purpose, workflow, features and team contributions of VRITTACARE.
10	Data Storage	Supabase PostgreSQL tables for profiles, assessments, prediction results and chat messages, filtered per user.
11	Machine Learning	Notebooks for data understanding, cleaning, exploratory analysis, feature engineering, model building and tuning. The preprocessor and final model are saved with joblib.
12	Backend API	FastAPI service with /health, /predict and /chat endpoints, Pydantic request validation, feature analysis and CORS configuration.
13	Deployment	Frontend and backend deployed as separate services on Render.

2.3 Project Plan (Gantt chart)

The project was carried out between 27 July 2026 and 18 October 2026. The work was divided into nine phases. Some phases ran in parallel, for example frontend development with backend work, and

documentation throughout the later part of the project. The dates below are based on the project's GitHub commit history.

Gantt chart:

Sr. No	Task Name	27 Jul	29 Jul	11 Aug	10 sep	21 Oct
1	Requirement Gathering					
2	Planning					
3	Designing					
4	Coding/Development					
5	Testing and Deployment					

Figure 1: *Gantt chart of the project plan (July – October 2026)*

Table 2.2: Project phases and schedule

Phase	Activity	Start	End	Days
P1	Project planning, dataset collection and data understanding	27 Jul 2026	29 Jul 2026	3
P2	Data cleaning and exploratory data analysis	29 Jul 2026	17 Aug 2026	20
P3	Feature engineering and preprocessing pipeline	07 Aug 2026	17 Aug 2026	11
P4	Model building and evaluation (four algorithms)	21 Aug 2026	23 Aug 2026	3
P5	Hyperparameter tuning and final model selection	23 Aug 2026	10 Sep 2026	19
P6	Feature analysis, Gemini assistant and FastAPI backend	12 Sep 2026	19 Sep 2026	8
P7	Frontend (VRITTACARE UI) and Supabase integration	11 Sep 2026	13 Oct 2026	33
P8	System integration, testing and Render deployment	09 Oct 2026	18 Oct 2026	10
P9	Documentation, screenshots and report preparation	02 Sep 2026	18 Oct 2026	47

3. PROJECT REQUIREMENT

3.1 Hardware Requirement

VRITTACARE runs in a web browser, so the end user needs only a basic device with an internet connection. The model training and notebook work needs a slightly more capable development computer. The requirements are listed below.

Development System (Minimum)

- Processor: Intel Core i3 / AMD Ryzen 3 or above (Core i5 or above recommended for training and notebooks).
- RAM: 8 GB (16 GB recommended).
- Hard disk: 256 GB SSD with at least 10 GB of free space.
- Input devices: Keyboard and mouse.
- Output device: Monitor with 1366 × 768 resolution or higher.
- Network: Stable internet connection for GitHub, Supabase, Render and the Gemini API.

End User (Client)

- Any laptop, desktop, tablet or smartphone with a modern web browser.
- Internet connection. Because the services run on Render's free plan, the first request after a period of inactivity may need a short warm-up time.

3.2 Software Requirement

The software used for development and deployment of the project is listed below. Versions of the Python packages are taken from the backend requirements file of the project.

Table 3.1: Software requirements

Category	Software / Library	Version / Details
Operating system	Windows 10 / 11	Development environment
Programming languages	Python, TypeScript (JavaScript)	Python for ML and API, TypeScript for the web application
Backend framework	FastAPI, Uvicorn, Pydantic	FastAPI 0.141.1, Uvicorn 0.52.4, Pydantic 2.13.5
Machine learning	scikit-learn, pandas, NumPy, joblib	scikit-learn 1.8.0, pandas 3.0.5, NumPy 2.5.3, joblib 1.6.0
Analysis and plotting	Jupyter Notebook, Matplotlib, Seaborn	Used for EDA and model development

Category	Software / Library	Version / Details
Frontend	React, TanStack Start / Router / Query, Vite	React 19, TanStack Start 1.x, Vite 8
Styling and charts	Tailwind CSS, Recharts, Zod	Tailwind CSS 4, Recharts 2, Zod 4
Authentication and database	Supabase (Auth, PostgreSQL)	supabase-js 2 (frontend), supabase 2.31.0 (backend)
AI assistant	Google Gemini API	google-genai 2.23.0
Deployment	Render	Separate web services for frontend and backend
Version control	Git and GitHub	Repository: Students_Mental_Health_Score_Prediction_sys
Browser	Google Chrome / Microsoft Edge / Firefox	Latest versions

Table 3.2: Main Python packages used by the backend

Package	Version	Purpose
Fastapi	0.141.1	REST API framework
Uvicorn	0.52.4	ASGI server used on Render
Pydantic	2.13.5	Request validation and schemas
scikit-learn	1.8.0	Random Forest model and preprocessing pipeline
Pandas	3.0.5	Data handling
Joblib	1.6.0	Loading the saved preprocessor and model
Supabase	2.31.0	Supabase client for the backend
google-genai	2.23.0	Google Gemini client for the Wellness Assistant
python-dotenv	1.2.3	Loading environment variables from .env
Requests	2.34.2	Downloading the model file when it is not present

3.3 Field Work

The field work of the project consisted of studying the problem domain, understanding the lifestyle and digital-habit variables that are associated with student well-being, selecting a suitable dataset and analysing it in detail before building the model. This section describes the dataset and the main findings of the analysis.

Dataset

The dataset used is “Student Social Media And Mental Health Impact”. It contains 5,000 records and 13 columns: twelve input variables and one target variable, the Mental Health Score. There are no missing values. The variables are described in Table 3.3.

Table 3.3: Data dictionary

Column	Type	Description / values
Age	Numeric	Age of the student, 18 to 24 years (mean 20.8)
Gender	Categorical	Female, Male
Country	Categorical	Country of the student (111 countries in the data)
Academic_Level	Categorical	High School, Undergraduate, Graduate
Most_Used_Platform	Categorical	12 platforms, e.g. Instagram, TikTok, YouTube, WhatsApp, Facebook, Snapchat
Purpose_Of_Use	Categorical	Education, Entertainment, Networking, News
Avg_Daily_Usage_Hours	Numeric	Average daily social media use, 1.0 to 8.8 hours (mean 5.08)
Daily_Unlocks	Numeric	Number of times the phone is unlocked per day, 62 to 273 (mean 171.5)
Study_Hours	Numeric	Daily study hours, 0.3 to 8.3 (mean 3.01)
Physical_Activity_Hours	Numeric	Daily physical activity, 0.0 to 4.1 hours (mean 1.75)
Sleep_Hours_Per_Night	Numeric	Sleep duration, 3.6 to 9.9 hours (mean 6.63)
Stress_Level	Ordinal	Low, Medium, High, Very High
Mental_Health_Score	Numeric (target)	Score from 3.6 to 9.4 (mean 6.23); higher means better wellbeing

Data Cleaning

- The dataset was checked for missing values (none found) and duplicate records. Two duplicate rows were removed, leaving 4,998 records.
- Ten records had a negative value for physical activity hours (minimum −0.4). These impossible values were replaced with 0.
- The Country column has 111 values and many appear only a few times. Countries that occur fewer than five times were grouped into a single “Rare_Country” category to reduce noise in the one-hot encoding.

Exploratory Data Analysis

Univariate, bivariate and correlation analysis were carried out. The main findings are given below.

- The Mental Health Score ranges from 3.6 to 9.4, with most students scoring between 5 and 7.
- More screen time (usage hours and phone unlocks) goes together with a lower score, while more sleep, study and physical activity go together with a higher score (Table 3.4).
- The score falls steadily as stress level increases from Low to Very High, with the median score moving from about 8 to about 5.

- Daily usage hours and daily unlocks are very strongly related to each other (0.96), which is one reason a tree-based model was a good choice.

Table 3.4: Correlation of each input with the Mental Health Score

Feature	Correlation	Direction
Avg_Daily_Usage_Hours	−0.816	Strong negative
Daily_Unlocks	−0.791	Strong negative
Sleep_Hours_Per_Night	+0.766	Strong positive
Study_Hours	+0.752	Strong positive
Physical_Activity_Hours	+0.522	Moderate positive
Age	+0.100	Weak positive

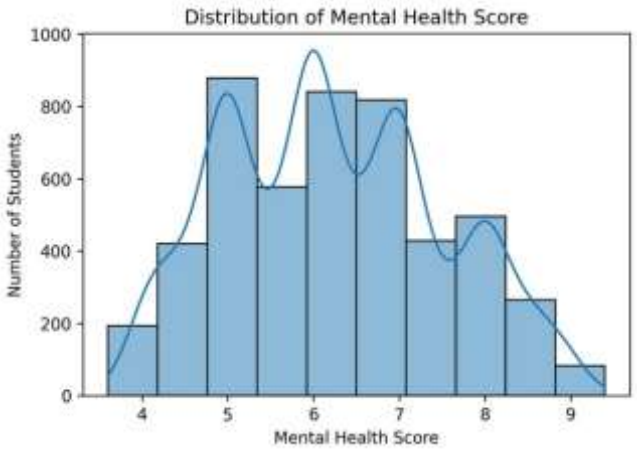


Figure 2: *Distribution of the Mental Health Score*

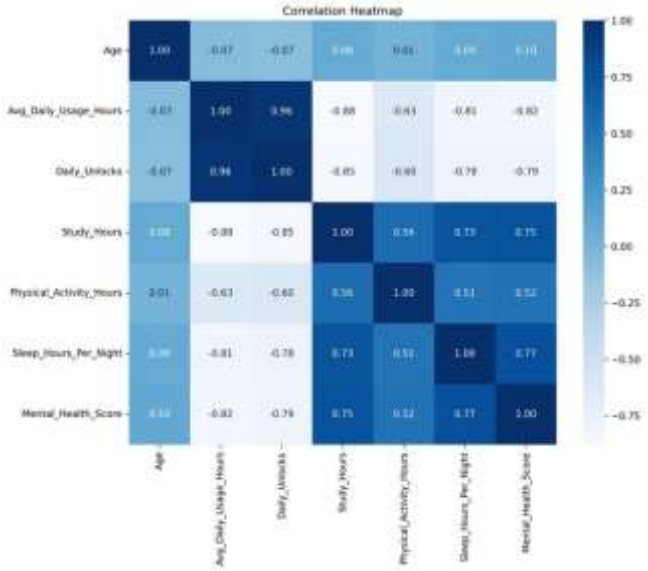


Figure 3: *Correlation heatmap of numeric variables*

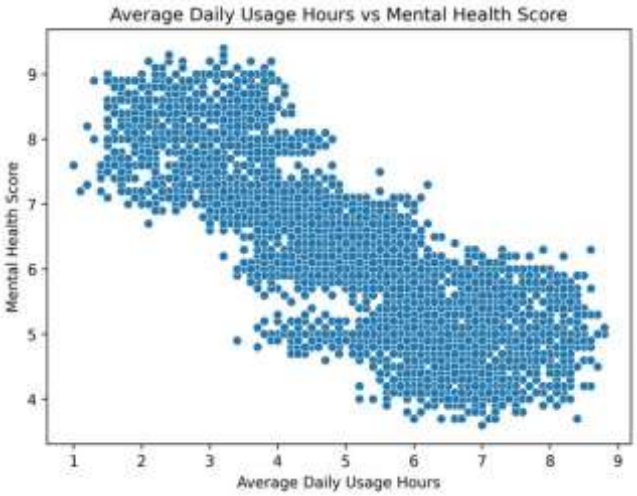


Figure 4: *Average daily usage hours vs Mental Health Score*

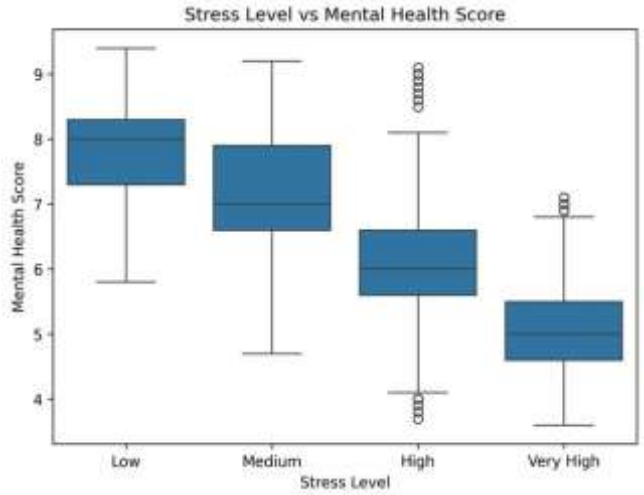


Figure 5: *Stress level vs Mental Health Score*

3.4 Functional and Non-Functional Requirements

The requirements of VRITTACARE are divided into functional requirements, which describe what the system must do, and non-functional requirements, which describe the quality of the system.

Functional requirements

- FR1 – Registration and sign-in: a student shall be able to create an account, sign in and sign out.
- FR2 – Assessment: the system shall collect twelve inputs about demographics, digital habits, lifestyle and stress.
- FR3 – Validation: the system shall reject missing or wrongly typed inputs before prediction.

- FR4 – Prediction: the system shall return a Mental Health Score between 0 and 10 for a valid assessment.
- FR5 – Categorisation: the system shall map the score to Low, Moderate, Good or Excellent / Stable.
- FR6 – Feature analysis: the system shall group each habit into Needs Attention, Can Improve or Favourable with an explanation.
- FR7 – Storage: the system shall store each assessment, result and chat message against the signed-in user.
- FR8 – Insights and dashboard: the system shall show the latest score, lifestyle metrics and practical next steps.
- FR9 – Assistant: the system shall answer the student's questions using the saved assessment and result.
- FR10 – Documentation and health check: the backend shall expose /health and /docs endpoints.

Non-functional requirements

Requirement	Description
Usability	Simple, calm and supportive screens; the assessment can be completed in one or two minutes.
Performance	A prediction is returned within a few seconds. The first request after inactivity may be slower on the free hosting plan.
Security	Authentication through Supabase Auth, user-scoped data with Row Level Security, secrets kept in environment variables.
Privacy	Only the data needed for the assessment is stored, and each student can access only his or her own records.
Reliability	The assessment is saved even if the prediction service is temporarily unavailable, and a clear message is shown.
Compatibility	Works in modern desktop and mobile browsers without installing any software.
Maintainability	Separate frontend, backend and notebook folders, typed schemas and documented API endpoints.
Responsible use	Every result is presented as an estimate for self-reflection and never as a diagnosis.

Use case diagram

The use case diagram shows how a student uses VRITTACARE. The prediction is produced by the ML model service, and the Wellness Assistant uses the Google Gemini API.

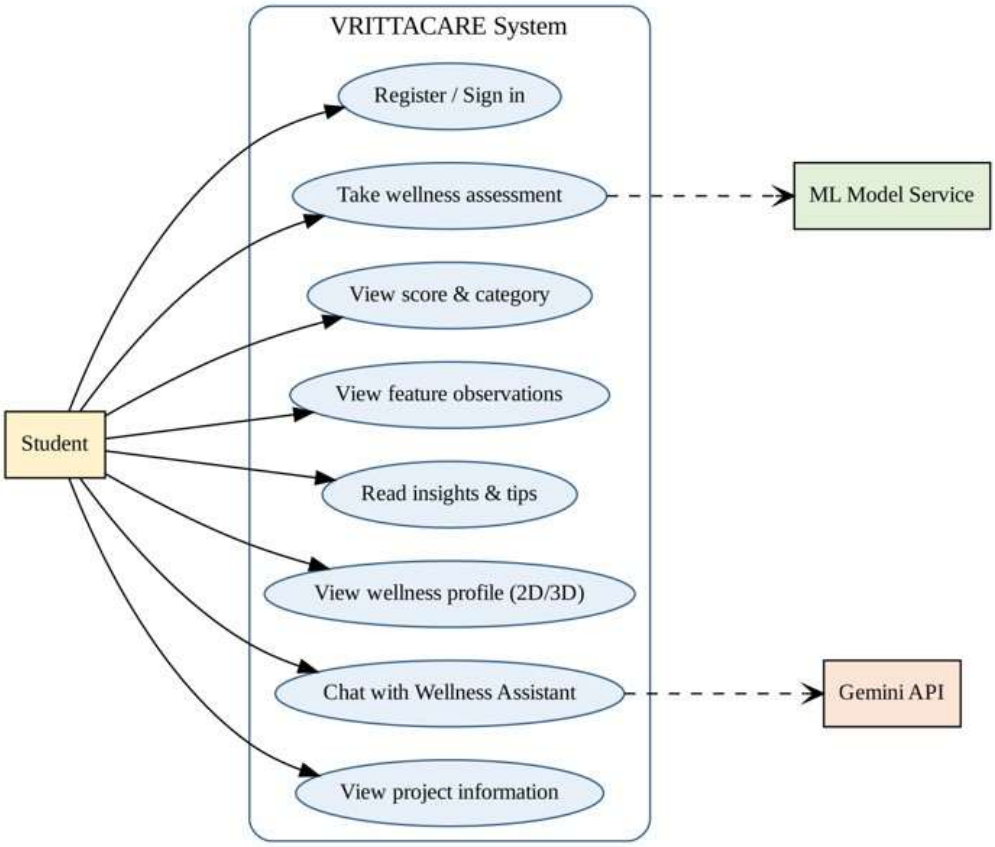


Figure 6: *Use case diagram of VRITTACARE*

Table 3.5: Use cases

No.	Use case	Description
UC1	Register / Sign in	The student creates an account or signs in with email and password or Google.
UC2	Take wellness assessment	The student enters the twelve inputs; the ML model service returns the prediction.
UC3	View score and category	The student sees the score gauge and its category.
UC4	View feature observations	Habits are shown as Needs Attention, Can Improve or Favorable.
UC5	Read insights and tips	The student reads a short focus path of practical next steps.
UC6	View wellness profile (2D / 3D)	Lifestyle signals are shown as bars and as an interactive 3D view.
UC7	Chat with Wellness Assistant	The student asks questions; replies are generated through the Gemini API.
UC8	View project information	The student reads the About page.

4. SYSTEM DESIGN

4.1 Database Design (Tables Schema)

VRITTACARE stores user data in a Supabase (PostgreSQL) database. Four tables are used. Every record belongs to a signed-in user through the user_id column, and the server functions always filter by the authenticated user, so a student can see only his or her own data. Sign-in accounts themselves are managed by Supabase Auth.

Table 4.1: profiles – one profile for every registered user.

Column	Data type	Key	Description
Id	uuid	PK, FK	Same as the Supabase Auth user id
display_name	text		Name shown on the dashboard
created_at	timestamp		Date and time of profile creation

Table 4.2: assessments – one row for every assessment submitted by a student.

Column	Data type	Key	Description
Id	uuid	PK	Unique id of the assessment
user_id	uuid	FK	Owner of the assessment
created_at	timestamp		Submission time
Age	integer		Age in years
Gender	text		Female / Male / Other
Country	text		Country of the student
academic_level	text		High School / Undergraduate / Graduate
most_used_platform	text		Social media platform used most
purpose_of_use	text		Main purpose of use
avg_daily_usage_hours	numeric		Average daily usage hours
daily_unlocks	integer		Phone unlocks per day
study_hours	numeric		Daily study hours
physical_activity_hours	numeric		Daily physical activity hours
sleep_hours_per_night	numeric		Sleep hours per night
stress_level	text		Low / Medium / High / Very High

Table 4.3: prediction_results – prediction returned by the model for an assessment.

Column	Data type	Key	Description
Id	uuid	PK	Unique id of the result
user_id	uuid	FK	Owner of the result
assessment_id	uuid	FK	Assessment that produced the result

Column	Data type	Key	Description
created_at	timestamp		Time of prediction
mental_health_score	numeric		Predicted score (10-point scale)
score_category	text		Display label of the score (Thriving / Stable / Mixed / Needs Care)
feature_analysis	jsonb		Feature importance, model version and the raw prediction response

Table 4.4: chat_messages – conversation history of the Wellness Assistant.

Column	Data type	Key	Description
Id	uuid	PK	Unique id of the message
user_id	uuid	FK	Owner of the conversation
Role	text		user or assistant
Message	text		Text of the message
created_at	timestamp		Time the message was saved

Note: PK = primary key, FK = foreign key (logical reference). Data types follow the PostgreSQL types used by the application.

4.2 E-R Diagram

The Entity-Relationship diagram shows the entities of the implemented system and the relationships between them. A user (managed by Supabase Auth) has one profile, submits many assessments, owns many prediction results and sends many chat messages. Every assessment produces one prediction result.

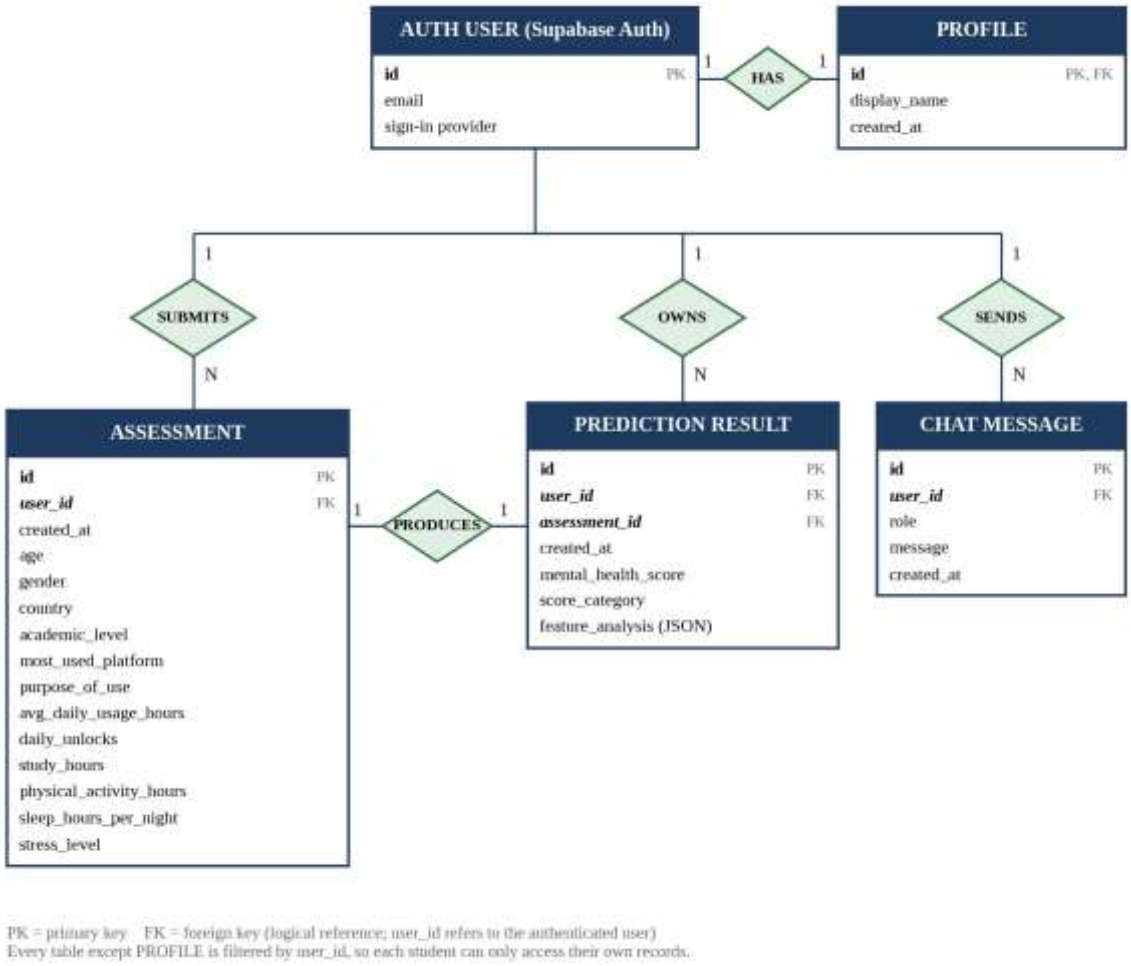


Figure 7: *E-R diagram of VRITTACARE*

Table 4.5: Relationships between entities

Entity 1	Entity 2	Relationship	Meaning
Auth User	Profile	1 : 1	Every user has one profile
Auth User	Assessment	1 : N	A user can submit many assessments
Assessment	Prediction Result	1 : 1	Every assessment produces one prediction result
Auth User	Prediction Result	1 : N	A user owns many prediction results
Auth User	Chat Message	1 : N	A user can send many chat messages

4.3 Data Flow Diagram (First Level)

The context-level diagram (Level 0) shows the system as a single process with two external entities: the **Student** and the **Google Gemini API**. The first-level diagram (Level 1) breaks the system into five processes and five data stores.



Figure 8: *Context-level data flow diagram (Level 0)*

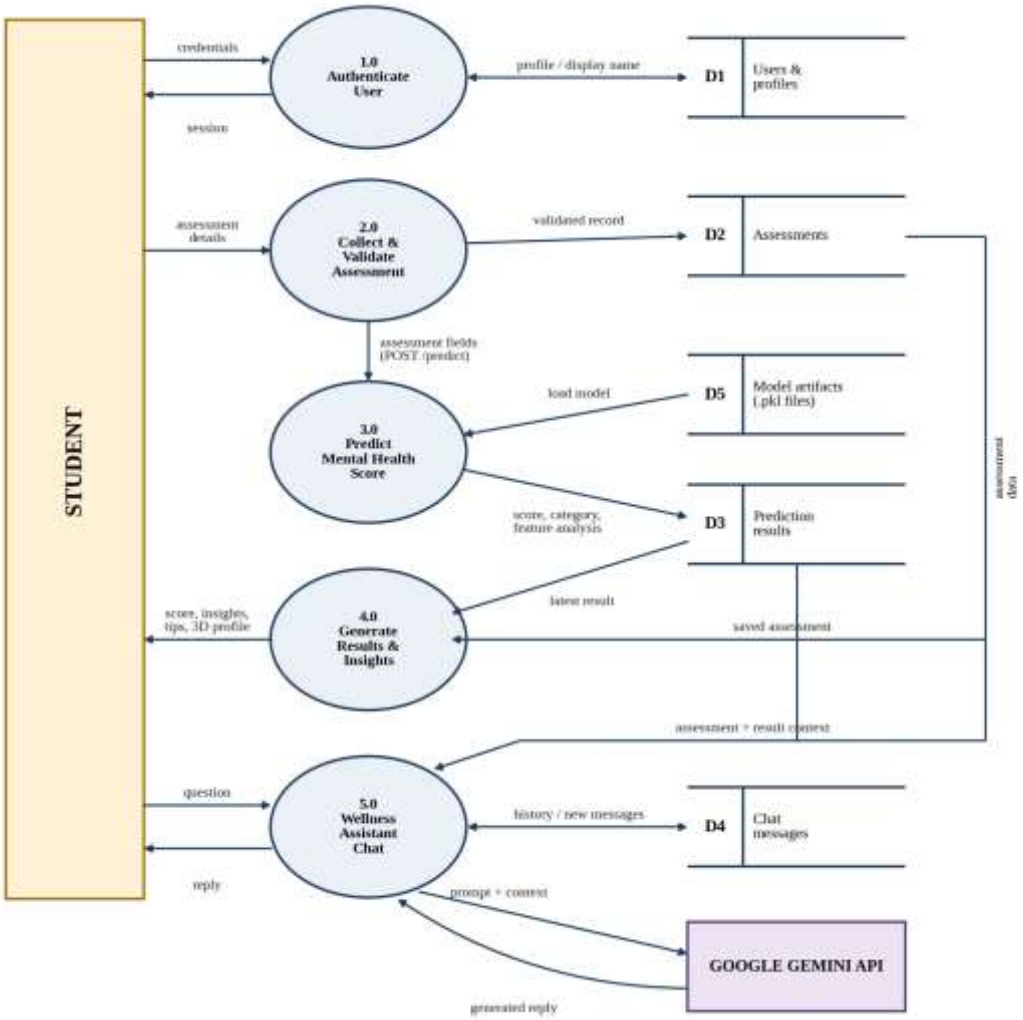


Figure 9: *First-level data flow diagram (Level 1)*

Table 4.6: Processes in the Level 1 diagram

Process	Name	Description
1.0	Authenticate User	Receives the student's credentials, creates or checks the account through Supabase Auth, stores the profile in D1 and returns a session.

Process	Name	Description
2.0	Collect and Validate Assessment	Receives the assessment details, validates every value against the allowed ranges and stores the validated record in D2.
3.0	Predict Mental Health Score	Sends the assessment to the FastAPI service, loads the preprocessor and Random Forest model (D5), predicts the score and category, creates the feature analysis and stores the result in D3.
4.0	Generate Results and Insights	Reads the saved assessment and result and shows the score gauge, wellness profile, insights, tips and 3D profile to the student.
5.0	Wellness Assistant Chat	Receives the student's question, reads the saved assessment, result and chat history, sends a prompt with context to the Gemini API, saves the messages in D4 and returns the reply.

Table 4.7: Data stores

Store	Name	Contents
D1	Users and profiles	Supabase Auth users and the profiles table
D2	Assessments	The assessments table
D3	Prediction results	The prediction_results table (score, category, feature analysis)
D4	Chat messages	The chat_messages table
D5	Model artifacts	preprocessor.pkl and final_random_forest.pkl used by the prediction service

5. DESIGNING

5.1 System Architecture

VRITTACARE follows a three-part architecture. The web application (frontend) runs in the student's browser and on its own server functions; a separate FastAPI service holds the machine-learning model; and Supabase stores user data. The Google Gemini API is used for the Wellness Assistant.

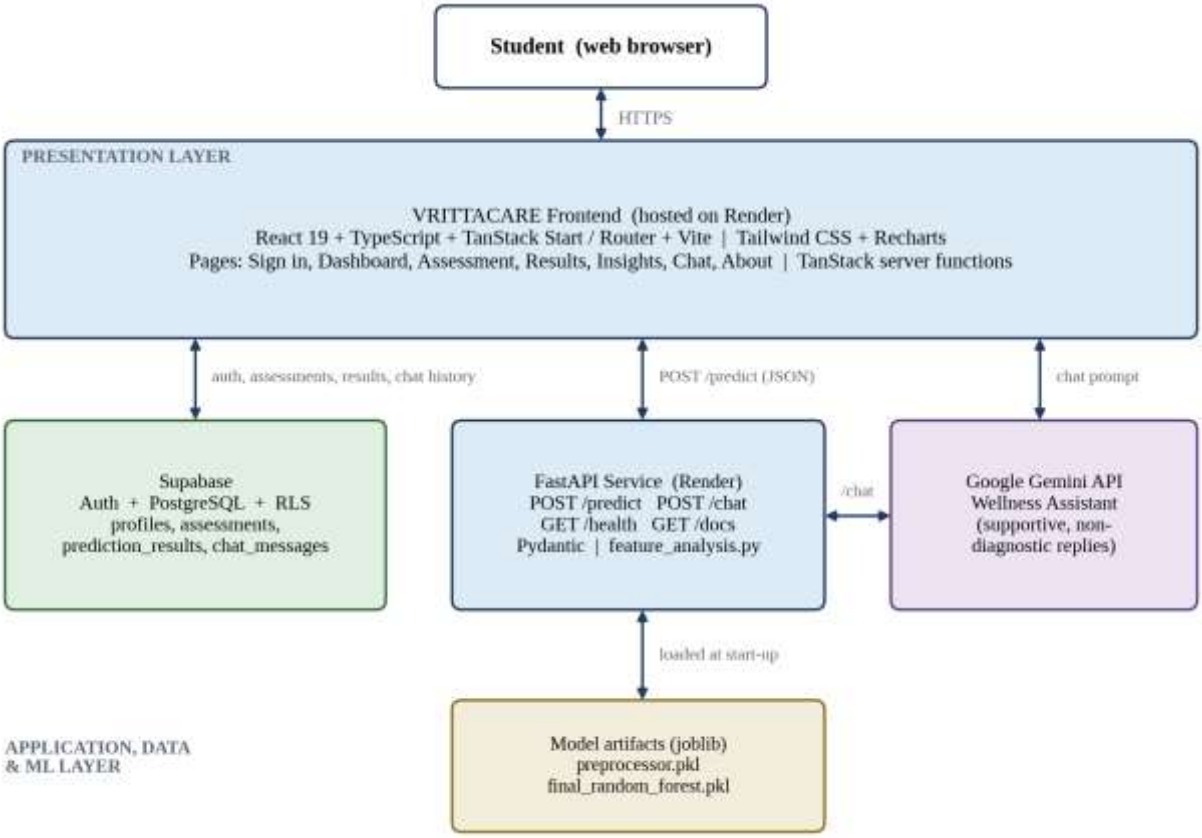


Figure 10: *System architecture of VRITTACARE*

End-to-end request flow

1. The student signs in and completes the assessment form.
2. The frontend server function validates the values and saves the assessment in Supabase.
3. The server function sends the assessment fields to the FastAPI /predict endpoint.
4. FastAPI validates the request with a Pydantic schema.
5. The backend applies the saved preprocessing transformer and the trained Random Forest model.
6. The backend returns the predicted score, the category and the grouped feature observations.
7. The result is saved in Supabase and shown as a score gauge, lifestyle summaries and insight cards.

8. When the student chats, the saved assessment, result and recent messages are sent to the Gemini API as context and the reply is saved in chat_messages.

5.2 Machine Learning Model Design

The machine-learning work was done in six notebooks that cover data understanding, cleaning, exploratory analysis, feature engineering, model building and model tuning. The figure below shows the overall workflow.

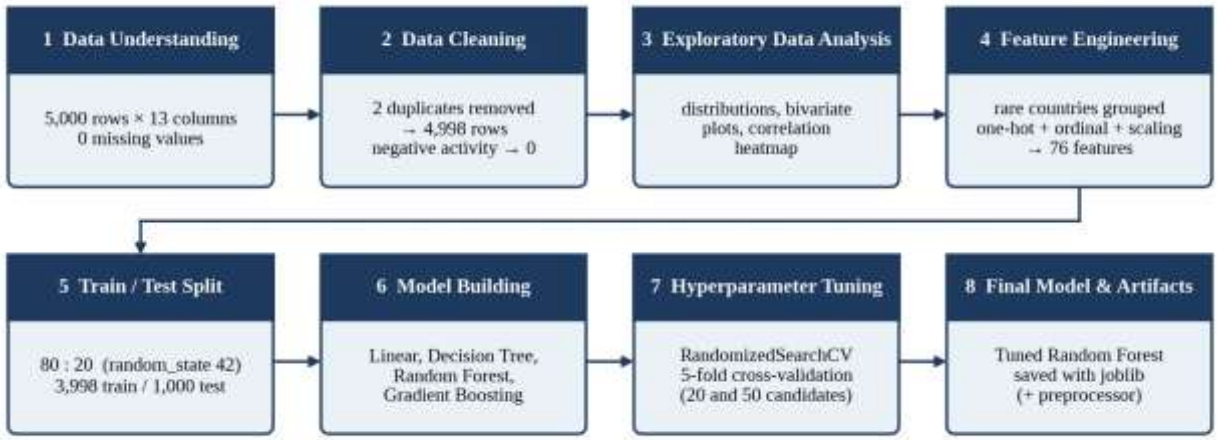


Figure 11: *Machine learning workflow*

Preprocessing

- Numeric inputs (age, usage hours, unlocks, study, activity, sleep) are standardised with a standard scaler.
- Stress level is an ordered variable and is encoded with an ordinal encoder (Low < Medium < High < Very High).
- Gender, academic level, platform, purpose and the processed country are one-hot encoded. Unknown categories seen at prediction time are ignored instead of causing an error.
- The target is Mental_Health_Score. The data was split 80:20 into **3,998 training** and **1,000 test** records with random_state 42. After preprocessing there are **76 input features**.

Model comparison

Four regression algorithms were trained on the same training data and compared on the test set using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the R^2 score. Random Forest gave the best results and was therefore selected for tuning.

Table 5.1: Model comparison on the test set

Model	MAE (lower is better)	RMSE (lower is better)	R^2 (higher is better)
Linear Regression	0.475815	0.613455	0.789197
Decision Tree Regressor	0.322800	0.515810	0.850964
Gradient Boosting Regressor	0.410558	0.529632	0.842870
Random Forest Regressor (baseline)	0.277949	0.385650	0.916690
Random Forest Regressor (tuned, final)	**0.267019**	**0.358643**	**0.927950**

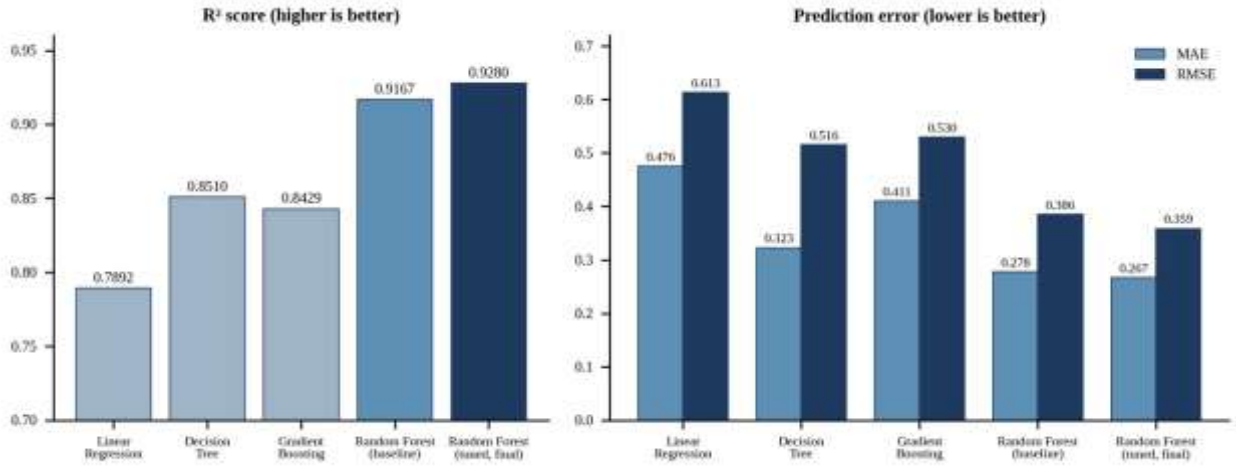


Figure 12: *Comparison of the models on the test set*

Hyperparameter tuning

The Random Forest was tuned with **RandomizedSearchCV** using five-fold cross-validation and R^2 as the scoring measure. A first search tested 20 combinations and a wider second search tested 50 combinations. The best cross-validated R^2 was **0.9033**, and the tuned model improved on the baseline in all three test measures.

Table 5.2: Final Random Forest hyperparameters

Hyperparameter	Selected value
n_estimators	500
max_features	log2
max_depth	None (trees grown fully)
min_samples_split	2
min_samples_leaf	1
random_state	42

Feature importance

The tuned model shows that screen-related habits, sleep, study time and stress contribute most to the prediction, while age and country contribute very little.

Table 5.3: Most important input features

Rank	Feature	Importance	Direction in data
1	Average daily usage hours	0.178	Higher usage, lower score
2	Daily phone unlocks	0.175	More unlocks, lower score
3	Sleep hours per night	0.164	More sleep, higher score
4	Study hours	0.121	More study, higher score
5	Stress level	0.118	Higher stress, lower score
6	Physical activity hours	0.062	More activity, higher score

5.3 Score Categories and Feature Analysis

After the score is predicted, the backend assigns a project-defined category and analyses every lifestyle input. These rules are interpretation aids made for this project. They are not clinical thresholds.

Table 5.4: Score categories

Score range	Category	Meaning
Below 4.0	Low	Lower estimated well-being pattern
4.0 to 6.9	Moderate	Mixed pattern; some areas can be improved
7.0 to 8.9	Good	Generally healthy pattern
9.0 and above	Excellent / Stable	Very favourable pattern

Each lifestyle input is placed in one of three groups. For areas that need attention, the model is re-run with a healthier reference value for that input and the difference is shown as an estimated score change, which helps the student see which change may matter most.

Table 5.5: Feature status rules

Input	Needs attention	Watch / can improve	Favorable
Daily screen usage (hours)	above 6.3	3.9 to 6.3	3.8 or less
Daily phone unlocks	above 100	51 to 100	50 or fewer
Study hours	1.5 or less	1.6 to 4.2	above 4.2
Physical activity (hours)	1.3 or less	1.4 to 2.2	above 2.2
Sleep (hours)	5.6 or less	5.7 to 7.5	above 7.5
Stress level	High, Very High	Medium	Low

Note: The unlock rule groups values above 100 as needing attention (above 200 is marked as high attention). The web application also shows a friendly display label on the score gauge (Thriving, Stable, Mixed or Needs Care); it is separate from the project categories above.

5.4 User Interface Design

The interface of VRITTACARE uses a calm dark theme with soft gradients and glass-style cards. The language on every screen is short, warm and non-judgemental, because the application is meant to support a student's self-reflection and not to label the student. The following pages show every screen of the application with its purpose and main elements.

Design principles

- Calm and non-clinical: a dark theme, soft gradients and glass cards create a relaxed feeling instead of a medical one.
- Supportive language: short and friendly sentences, with no medical terms and no judgement.
- One clear action per screen: each screen has one main button, so the student always knows the next step.
- Gradual detail: the score gauge comes first and the explanations follow, so the student is not overloaded.
- Consistent navigation: the same sidebar and the same floating assistant button appear on every page.
- Responsible communication: every result is described as an estimate for self-reflection and never as a diagnosis.

Screens at a glance

Screen	What it shows
Sign in and account creation	Email, Google and account creation
Home screen	Personal welcome and main navigation
Personalised dashboard	Latest score gauge and wellness profile
Wellness profile and next steps	Three suggested small steps
Wellness assessment	Twelve-input check-in form
Prediction results	Score, category and focus area
Insights and tips	What shaped the result
Focus path	Start here, then notice, keep doing
Practical next steps	Small steps and wellness rhythms
3D wellness profile	Interactive layered view of five signals

Screen	What it shows
Wellness Assistant	Supportive chat about the result
About the project	Purpose and responsible use
How it works and team	Workflow and project team

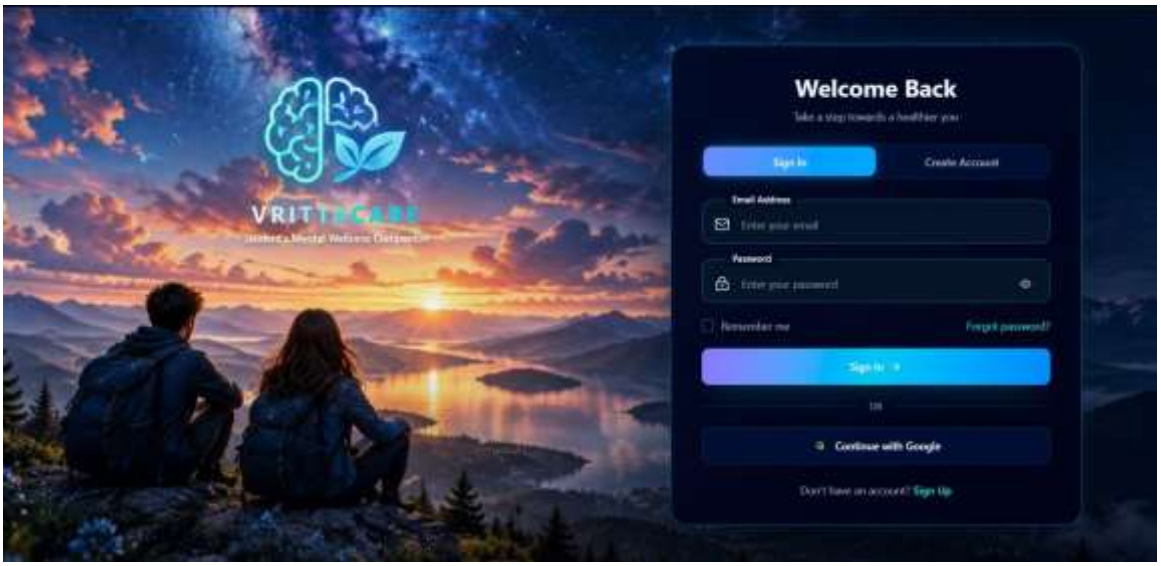


Figure 13: *Sign in and account creation*

The entry screen welcomes the student with a calm night-sky background and a glass-style card for signing in or creating an account.

- Sign In and Create Account tabs with email and password fields
- Continue with Google for one-click access
- Remember me and Forgot password options

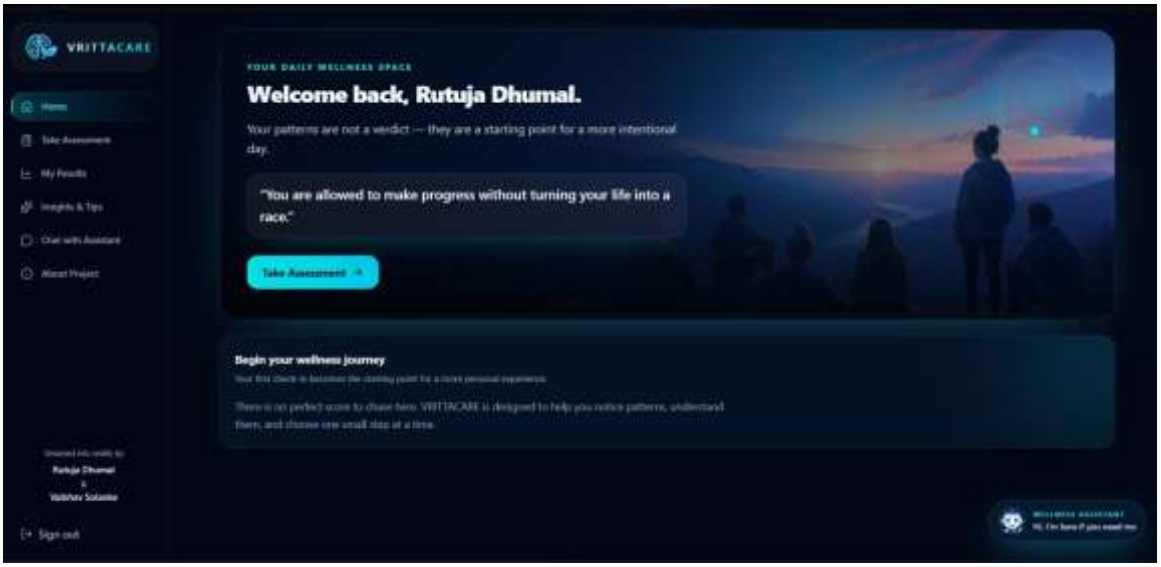


Figure 14: *Home screen*

After sign-in, the home screen greets the student by name with an encouraging quote and one clear action: start a check-in.

- Take Assessment as the main call to action
- Sidebar: Home, Assessment, Results, Insights, Chat, About
- Floating Wellness Assistant button on every page



Figure 15: *Personalised dashboard*

The dashboard is a snapshot of the latest check-in. It combines the predicted score with the main lifestyle signals.

- Score gauge with its category (Moderate in this example)
- Wellness Profile bars for sleep, study, activity, screen usage and stress
- Check in again, next step and 3D View buttons



Figure 16: *Wellness profile and next small steps*

The lower part of the dashboard turns the result into action with three small next steps, ordered by usefulness.

- Start here, then notice, keep doing

- Short and gentle step titles
- Profile bars for a quick comparison of the signals

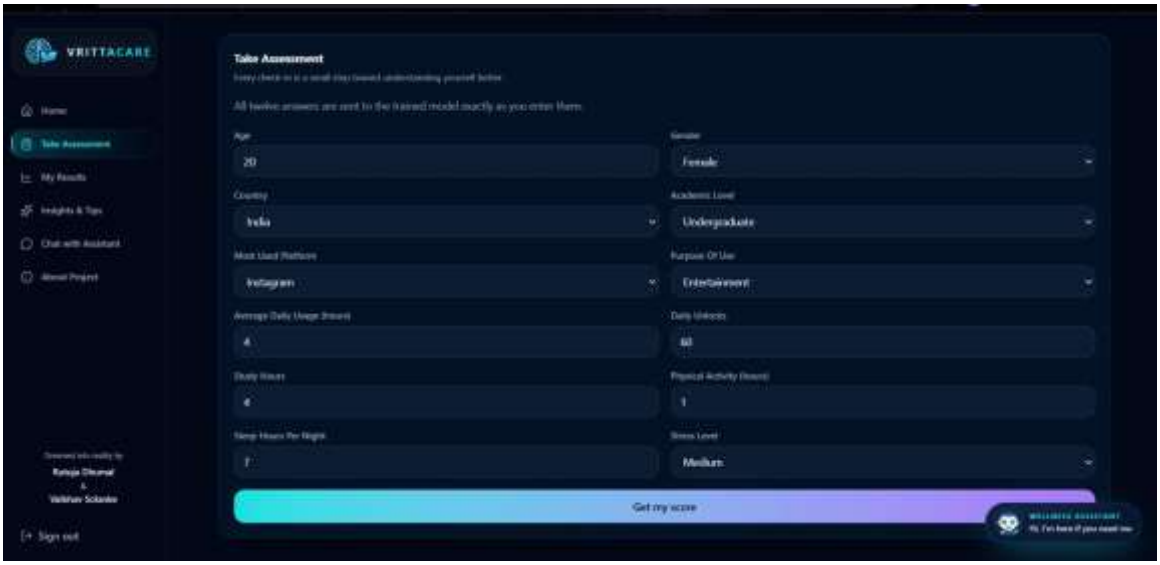


Figure 17: *Wellness assessment*

A short guided form collects twelve everyday details in simple language, so a check-in takes only a minute or two.

- Age, gender, country and academic level
- Platform, purpose, usage hours, unlocks, study, activity, sleep and stress
- Get my score submits the check-in



Figure 18: *Prediction results*

The Random Forest output is shown on a gauge with its category, so the student never sees a bare number.

- Score out of 10 with its category
- Next useful focus card linking to Insights

- Status tag for every lifestyle signal

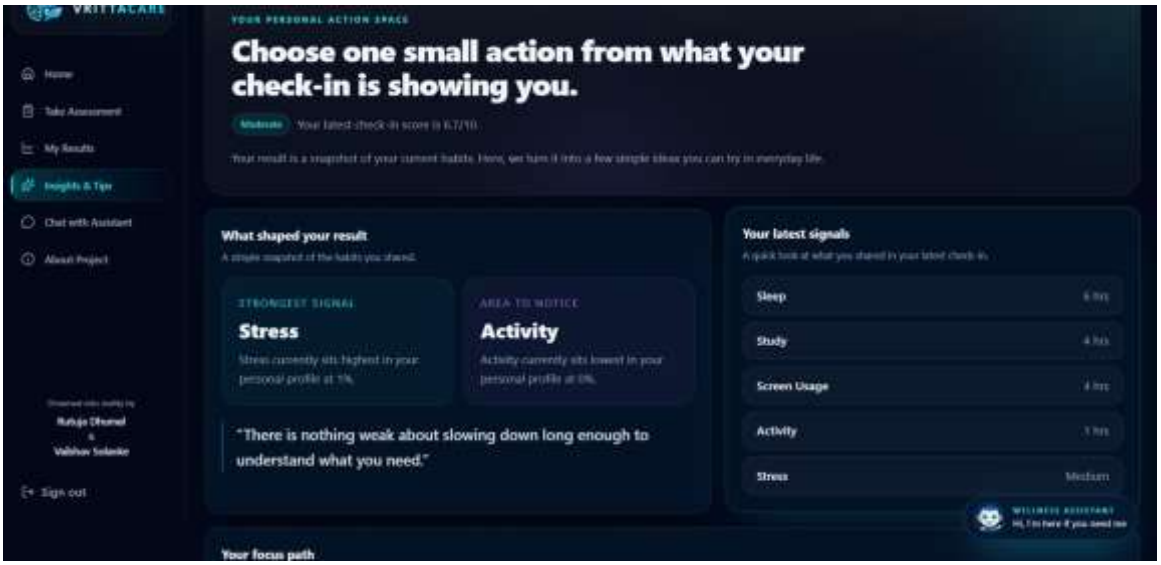


Figure 19: *Insights and tips*

The page explains what shaped the result and invites the student to choose one small action.

- Strongest signal and the area to notice
- A supportive, non-judgemental quote
- Latest signals with the values the student entered

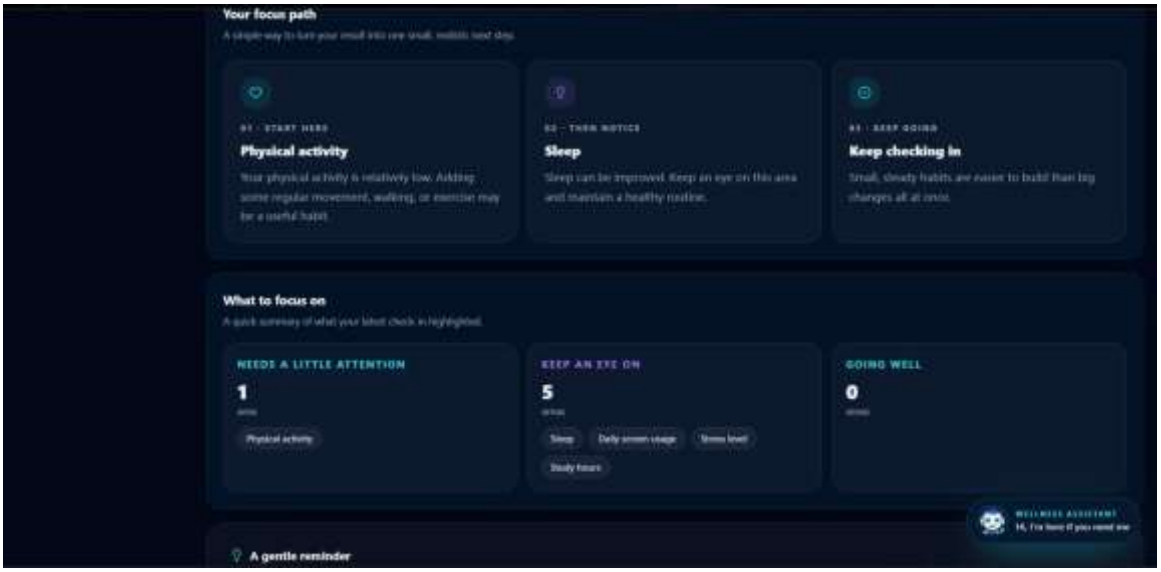


Figure 20: *Focus path*

A simple order of work: where to start, what to notice next and what to keep doing.

- Three-step focus path
- Counts for needs attention, keep an eye on and going well
- A gentle reminder and a link to the assistant

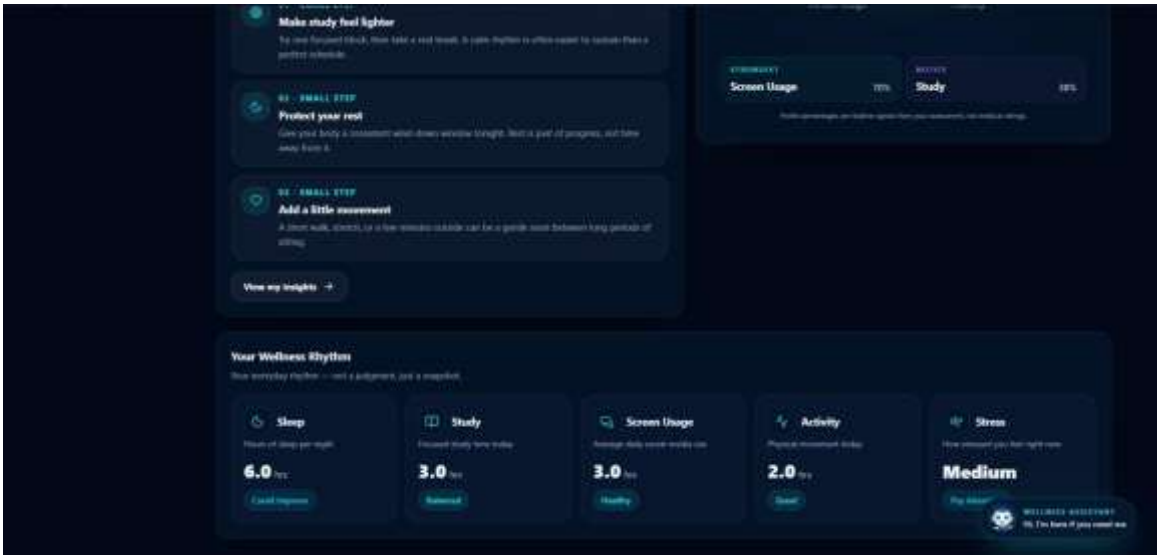


Figure 21: *Practical next steps*

Practical small steps and a summary of the student's wellness rhythms, designed to be tried within a day.

- Steps for activity, study and screen use
- Wellness Rhythms cards at a glance
- Button to open the full insights

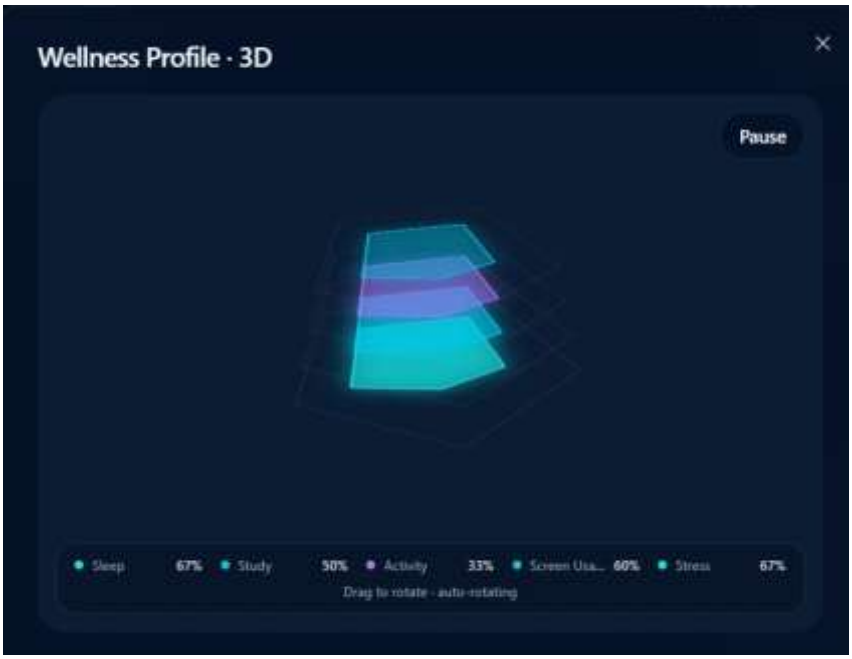


Figure 22: *3D wellness profile*

An interactive 3D view shows the five lifestyle signals as stacked layers, so their balance is easy to see.

- Layers for sleep, study, activity, screen usage and stress
- Drag to rotate, automatic rotation and a Pause button
- Percentage legend at the bottom



Figure 23: *Wellness Assistant*

The assistant explains the result in supportive language and answers follow-up questions.

- Greeting by name and an explanation of the latest score
- Quick suggestion buttons for common questions
- Reminder that a trusted person or professional can help



Figure 24: *About the project*

The About page explains what VRITTACARE is and what it is not.

- A student-built Mental Health Prediction System
- For self-reflection, not diagnosis
- Journey: answers, prediction, understanding, actions

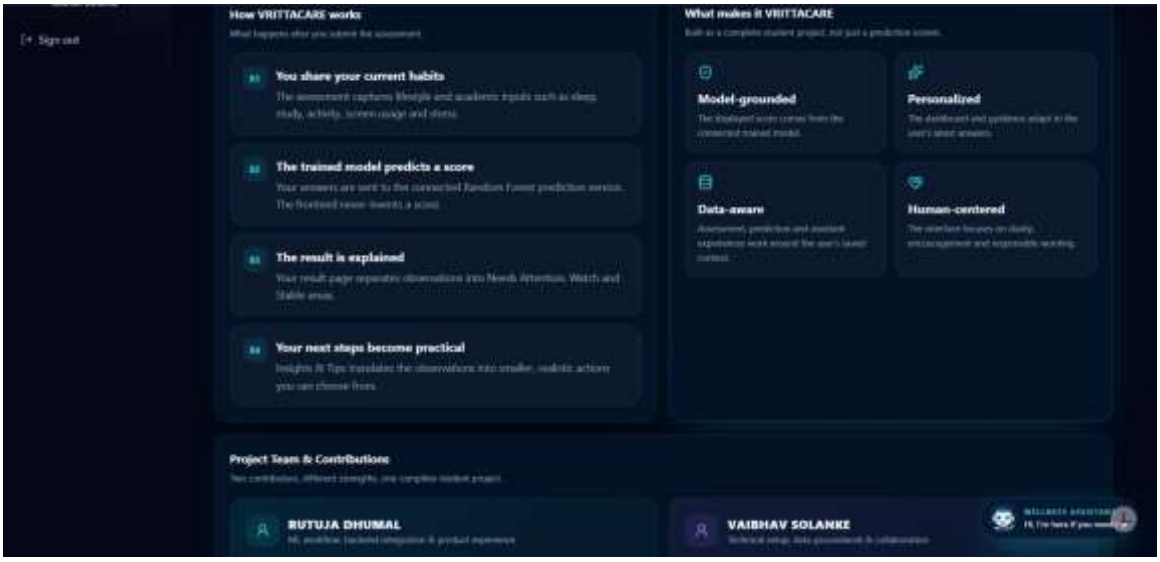


Figure 25: *How VRITTACARE works and project team*

The second half of the About page explains how the application works and who built it.

- Four-step workflow of the application
- What makes VRITTACARE different
- Project team and their contributions

Table 5.6: Pages of the application

Page	Route file	Purpose
Sign In / Create Account	routes/auth.tsx	Authentication with Supabase (email / password and Google)
Dashboard (Home)	routes/_authenticated/dashboard.tsx	Latest score, wellness profile, lifestyle rhythm and quick actions
Take Assessment	routes/_authenticated/assessment.tsx	Twelve-question form that saves the assessment and requests a prediction
My Results	routes/_authenticated/results.tsx	Score gauge, category and feature observations
Insights & Tips	routes/_authenticated/insights.tsx	Personalised action suggestions and focus path
Chat with Assistant	components/mb/assistant-panel.tsx	Gemini-powered Wellness Assistant panel
About Project	routes/_authenticated/about.tsx	Purpose, workflow and team of the project

5.5 Security, Privacy and Responsible-Use Design

VRITTACARE handles personal lifestyle information, so security and responsible use were considered in the design from the beginning.

Area	Measure in VRITTACARE
Authentication	Supabase Auth with email and password or Google. All pages except sign-in are available only to a signed-in student.
User-scoped data	Every record is stored with the owner's user_id. Server functions use the authenticated user, and Row Level Security applies to each operation.
Input validation	Zod ranges are checked in the web application and Pydantic types are checked in the FastAPI service.
Secrets and keys	Gemini, Supabase service-role and other keys are kept in environment variables, never in source code or Git.
Responsible wording	Every result is an estimate for self-reflection. Categories and feature rules are project-defined and not clinical thresholds.
Assistant behaviour	The assistant is instructed to be supportive and non-diagnostic and to encourage speaking to a trusted person or professional.

5.6 Limitations and Mitigation

The main limitations of VRITTACARE and the way the project handles or plans to handle them are listed below.

Limitation	Explanation	Mitigation / plan
Dataset based	One dataset of 4,998 records; it may not represent every student or country.	Results are shown as estimates; a larger, more diverse dataset is planned.
Correlation, not causation	The model finds associations and does not prove that a habit causes a change.	Careful wording such as may be a useful area to focus on.
Unseen categories and ranges	Other genders, extra platforms and values outside the training range (for example below 62 unlocks) were not seen in training.	Unknown categories are ignored without error; aligning the options with the training data is planned.
Project-defined rules	Categories and feature status rules are not validated clinical cut-offs.	Clearly labelled as guidance, not diagnosis.
Hosting and CORS	Free hosting can be slow after inactivity, and CORS is open for development.	Warm-up time is documented; CORS will be restricted to the frontend address.

6. CODING

This chapter presents the important parts of the source code of VRITTACARE. The complete project is available in the GitHub repository https://github.com/rutuu0228/Students_Mental_Health_Score_Prediction_sys. The listings are shortened where necessary, but every listing is taken directly from the repository files named in its caption.

6.1 Project Structure

The project is organised into separate frontend, backend, data, notebook, model, documentation and screenshot folders. This keeps the machine-learning workflow apart from the web application while allowing both to work together.

```
Students_Mental_Health_Score_Prediction_sys/
|-- Backend/                                  # FastAPI service + ML inference
|   |-- api/
|   |   |-- main.py                           # FastAPI app, CORS, /health
|   |   |-- schemas.py                        # Pydantic request schemas
|   |   `-- routes/
|   |       |-- prediction.py                 # POST /predict
|   |       `-- chatbot.py                    # POST /chat
|   |-- data/cleaned_data.csv
|   |-- models/                               # preprocessor.pkl, final_random_forest.pkl
|   |-- predictor.py                          # model loading and prediction
|   |-- feature_analysis.py                   # categories, feature rules, what-if analysis
|   |-- gemini_chatbot.py                     # Gemini Wellness Assistant
|   |-- supabase_client.py
|   |-- requirements.txt
|   `-- Procfile                              # Render start command
|-- Frontend/                                 # React + TypeScript (TanStack Start)
|   |-- src/routes/                           # pages (auth, dashboard, assessment, results ...)
|   |-- src/components/mb/                    # gauge, radar chart, assistant panel ...
|   |-- src/lib/                              # prediction.ts, dashboard.functions.ts, mb.ts ...
|   |-- src/integrations/supabase/
|   `-- package.json
|-- Data/                                     # raw, cleaned and processed datasets
|-- Notebooks/                                # 01 - 06 data understanding to model tuning
|-- Models/                                   # saved preprocessor and model
|-- Images/                                   # EDA and model plots
|-- Documentation/                            # synopsis, diagrams, presentation
|-- VRITTACARE_screenshots/
`-- README.md
```

6.2 Backend Code

6.2.1 FastAPI application, schemas and routes

The application object, CORS configuration and the health-check endpoint are defined in api/main.py. The request body of the prediction endpoint is described by the StudentInput Pydantic model. FastAPI uses it to validate the types of all twelve fields automatically and rejects bad requests with a clear error before the model is called.

Listing 6.1: Backend/api/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from api.routes.prediction import router as prediction_router
from api.routes.chatbot import router as chatbot_router

app = FastAPI(
    title="Mental Health Prediction System"
)

# Allow the Lovable frontend and local development to call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
def health_check():
    return {"status": "ok"}

app.include_router(prediction_router)
app.include_router(chatbot_router)
```

Listing 6.2: Backend/api/schemas.py

```
from pydantic import BaseModel

class StudentInput(BaseModel):
    Age: int
    Gender: str
    Academic_Level: str
    Most_Used_Platform: str
    Purpose_Of_Use: str
    Avg_Daily_Usage_Hours: float
    Daily_Unlocks: int
    Study_Hours: float
    Physical_Activity_Hours: float
    Sleep_Hours_Per_Night: float
    Stress_Level: str
```

```
    Country: str

class ChatInput(BaseModel):
    message: str
```

Listing 6.3: Backend/api/routes/prediction.py

```
from fastapi import APIRouter

from api.schemas import StudentInput
from predictor import create_prediction_context

router = APIRouter(
    prefix="/predict",
    tags=["Prediction"]
)

@router.post("")
def predict(student: StudentInput):

    student_data = student.model_dump()

    prediction_context = create_prediction_context(
        student_data
    )

    return prediction_context
```

Listing 6.4: Backend/api/routes/chatbot.py

```
from fastapi import APIRouter

from api.schemas import ChatInput
from gemini_chatbot import chat_with_gemini

router = APIRouter(
    prefix="/chat",
    tags=["Chatbot"]
)

@router.post("")
def chat(chat_input: ChatInput):

    response = chat_with_gemini(
        chat_input.message
    )

    return {
        "response": response
    }
```

6.2.2 Loading the model and predicting

predictor.py loads the cleaned dataset (to know how often each country occurs), the saved preprocessing pipeline and the final Random Forest model. If the model file is not present on the server, it is downloaded once from the URL given by the MODEL_URL environment variable, which keeps the large model file out of the normal Git history. predict_mental_health_score() groups rare

countries exactly as in training, applies preprocessor.transform() and returns the model output rounded to two decimals.

Listing 6.5: Backend/predictor.py – model loading and prediction

```
BASE_DIR = Path(__file__).resolve().parent

CLEANED_DATA_PATH = BASE_DIR / "data" / "cleaned_data.csv"
PREPROCESSOR_PATH = BASE_DIR / "models" / "preprocessor.pkl"
MODEL_PATH = BASE_DIR / "models" / "final_random_forest.pkl"

MODEL_URL = os.environ.get(
    "MODEL_URL",
    "https://github.com/rutuu0228/students-mental-health-
backend/releases/download/v1.0.0/final_random_forest.pkl",
)

cleaned_data = pd.read_csv(CLEANED_DATA_PATH)
country_counts = cleaned_data["Country"].value_counts()

if not MODEL_PATH.exists():
    MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)

    response = requests.get(MODEL_URL, timeout=120)
    response.raise_for_status()

    with open(MODEL_PATH, "wb") as file:
        file.write(response.content)

preprocessor = joblib.load(PREPROCESSOR_PATH)
model = joblib.load(MODEL_PATH)

def predict_mental_health_score(student_data):
    input_df = pd.DataFrame([student_data])

    input_df["Country_Processed"] = input_df["Country"].apply(
        lambda x: "Rare_Country"
        if country_counts.get(x, 0) < 5
        else x
    )

    input_df = input_df.drop(columns=["Country"])

    processed_data = preprocessor.transform(input_df)
    prediction = model.predict(processed_data)[0]

    return round(float(prediction), 2)

def create_prediction_context(student_data):
    score = predict_mental_health_score(student_data)

    analysis = analyze_student_features(
        student_data,
        model=model,
        preprocessor=preprocessor,
    )
```

```
    ai_context = create_ai_feature_context(
        student_data,
        score,
        analysis,
    )

    ai_context["category"] = get_score_category(score)

    return ai_context
```

6.2.3 Score category and feature analysis

feature_analysis.py contains the project-defined score categories, the rules that decide whether a habit needs attention or is favourable, and the what-if calculation. The first excerpt shows get_score_category() and the rule for sleep. The second shows analyze_student_features(), in which a counterfactual copy of the student's data is created, the model is run again, and the difference in score is stored as estimated_score_change.

Listing 6.6: Backend/feature_analysis.py – category and sleep rule

```
def get_score_category(score):
    if score < 4.0:
        return "Low"
    elif score < 7.0:
        return "Moderate"
    elif score < 9.0:
        return "Good"
    else:
        return "Excellent / Stable"

# ... other features follow the same pattern ...

    "Sleep_Hours_Per_Night": {
        "label": "Sleep",
        "direction": "higher_is_better",
        "attention": lambda x: x <= 5.6,
        "stable": lambda x: x > 7.5,
        "reference": 8.2,
        "attention_message": (
            "Your sleep duration is relatively low. "
            "Gradually working toward a more regular and "
            "sufficient sleep routine may be helpful."
        ),
        "stable_message": (
            "Your sleep duration is in a favorable range. "
            "Try to keep a consistent sleep routine."
        ),
    },
```

Listing 6.7: Backend/feature_analysis.py – status and what-if analysis

```
def get_feature_status(feature, value):
    if feature == "Daily_Unlocks":
        return classify_daily_unlocks(value)

    rule = FEATURE_RULES[feature]
```

```
    if rule["attention"](value):
        return {
            "status": "attention",
            "interpretation": "Needs attention",
        }
    elif rule["stable"](value):
        return {
            "status": "stable",
            "interpretation": "Favorable",
        }
    else:
        return {
            "status": "watch",
            "interpretation": "Can improve",
        }

def analyze_student_features(student_data, model=None, preprocessor=None):
    needs_attention = []
    stable = []
    watch = []
    base_score = None
    if model is not None and preprocessor is not None:
        input_df = prepare_model_input(student_data)
        processed_data = preprocessor.transform(input_df)
        base_score = float(model.predict(processed_data)[0])

    for feature, rule in FEATURE_RULES.items():
        if feature not in student_data:
            continue

        value = student_data[feature]

        item = {
            "feature": feature,
            "label": rule["label"],
            "value": value,
        }

        if base_score is not None and "reference" in rule:
            counterfactual = student_data.copy()
            counterfactual[feature] = rule["reference"]

            cf_df = prepare_model_input(counterfactual)
            cf_processed = preprocessor.transform(cf_df)
            cf_score = float(model.predict(cf_processed)[0])

            item["estimated_score_change"] = round(
                cf_score - base_score,
                2,
            )

        status = get  feature  status(feature, value)

        item["status"] = status["status"]
        item["interpretation"] = status["interpretation"]

        # ... the item is added to needs_attention, watch or stable ...
```

6.2.4 Wellness Assistant (Gemini)

gemini_chatbot.py creates a Gemini client from the GEMINI_API_KEY environment variable. For every message it builds a context text (name, score, category, the three feature groups and the question) and calls the Gemini API. The first message in a conversation also carries the long system instruction; later messages continue the same interaction through previous_interaction_id.

Listing 6.8: Backend/gemini_chatbot.py – client setup

```
import os

from dotenv import load_dotenv
from google import genai

# ============================================================
# GEMINI SETUP
# ============================================================

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

previous_interaction_id = None
```

Listing 6.9: Backend/gemini_chatbot.py – calling Gemini

```
    # ----------------------------------------------------
    # Gemini interaction
    # ----------------------------------------------------

    if previous_interaction_id is None:

        response = client.interactions.create(

            model="gemini-3.6-flash",

            input=(
                SYSTEM_INSTRUCTION
                + "\n\n"
                + user_input
            )
        )

    else:

        response = client.interactions.create(

            model="gemini-3.6-flash",

            input=user_input,

            previous_interaction_id=
                previous_interaction_id
        )
```

```
# Save conversation
previous_interaction_id = response.id
```

6.2.5 Example request and response

The following request is sent to POST /predict. The response contains the score, the category and the three groups of observations. The response values shown here are illustrative and only demonstrate the structure; real values depend on the input.

Listing 6.10: Example request (illustrative values)

```
{
  "Age": 20,
  "Gender": "Female",
  "Academic_Level": "Undergraduate",
  "Most_Used_Platform": "Instagram",
  "Purpose_Of_Use": "Entertainment",
  "Avg_Daily_Usage_Hours": 4.5,
  "Daily_Unlocks": 60,
  "Study_Hours": 3.0,
  "Physical_Activity_Hours": 1.0,
  "Sleep_Hours_Per_Night": 7.0,
  "Stress_Level": "Medium",
  "Country": "India"
}
```

Listing 6.11: Structure of the /predict response (illustrative values)

```
{
  "score": 6.7,
  "category": "Moderate",
  "needs_attention": [
     { "area": "Physical activity", "current_value": 1.0,
       "interpretation": "Needs attention",
       "message": "Your physical activity is relatively low. ...",
       "estimated_score_change": 0.4 }
  ],
  "watch": [ { "area": "Sleep", "current_value": 7.0, "interpretation": "Can improve", ... } ],
  "stable": [ { "area": "Daily screen usage", "current_value": 3.0, "interpretation": "Favorable", ... } ]
}
```

6.3 Machine-Learning Code

The notebooks in the Notebooks folder document the complete machine-learning workflow. The excerpts below show the key steps: cleaning and grouping rare countries, building the preprocessing pipeline, splitting the data, training the Random Forest, tuning it with RandomizedSearchCV and saving the final model. A second, wider search of 50 candidates with the same code pattern produced the final settings listed in Table 5.2.

Listing 6.12: Notebooks 02 and 04 – cleaning and rare-country grouping

```
# 02_Data_Cleaning: remove duplicates and fix invalid values
df.drop_duplicates(inplace=True)
df.loc[df['Physical_Activity_Hours'] < 0, 'Physical_Activity_Hours'] = 0

# 04_Feature_Engineering: group rare countries
country_counts = df["Country"].value_counts()
df["Country_Processed"] = df["Country"].apply(
    lambda x: "Rare_Country" if country_counts[x] < 5 else x
)
```

Listing 6.13: Notebook 04 – preprocessing pipeline

```
preprocessor = ColumnTransformer(
    transformers=[
        ("onehot",
         OneHotEncoder(handle_unknown="ignore", sparse_output=False),
         one_hot_features),
        ("ordinal",
         OrdinalEncoder(categories=stress_order),
         ordinal_features),
        ("scaler",
         StandardScaler(),
         numerical_features)
    ],
    remainder="drop"
)

joblib.dump(preprocessor, "../Models/preprocessor.pkl")
```

Listing 6.14: Notebook 05 – train/test split and Random Forest

```
X = df.drop(columns=["Mental_Health_Score"])
y = df["Mental  Health  Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

random_forest_model = RandomForestRegressor(random_state=42)
random_forest_model.fit(X_train, y_train)
y_pred_rf = random_forest_model.predict(X_test)

rf_mae = mean_absolute_error(y_test, y_pred_rf)
rf_rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
rf_r2 = r2_score(y_test, y_pred_rf)
```

Listing 6.15: Notebook 06 – hyperparameter tuning and saving

```
param_grid = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [None, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", None]
}

rf_random_search = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_distributions=param_grid,
```

```
    n_iter=20, cv=5, scoring="r2",
    random_state=42, n_jobs=-1
)
rf_random_search.fit(X_train, y_train)

best_rf = rf_random_search.best_estimator_
joblib.dump(best_rf, "../Models/final_random_forest.pkl")
```

6.4 Frontend Code

6.4.1 Authentication and protected routes

The useAuth hook listens to Supabase authentication changes and exposes the current session and user. All pages after sign-in are placed under the _authenticated layout route. Its beforeLoad function asks Supabase for the current user and redirects to /auth when nobody is signed in.

Listing 6.16: Frontend/src/hooks/use-auth.ts

```
import { useEffect, useState } from "react";
import type { Session, User } from "@supabase/supabase-js";

import { supabase } from "@/integrations/supabase/client";

export function useAuth() {
  const [session, setSession] = useState<Session | null>(null);
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const { data: sub } = supabase.auth.onAuthStateChange((_event, next) => {
      setSession(next);
      setUser(next?.user ?? null);
      setLoading(false);
    });

    supabase.auth.getSession().then(({ data }) => {
      setSession(data.session);
      setUser(data.session?.user ?? null);
      setLoading(false);
    });

    return () => sub.subscription.unsubscribe();
  }, []);

  return { session, user, loading };
}
```

Listing 6.17: Frontend/src/routes/_authenticated/route.tsx

```
import { createFileRoute, Outlet, redirect } from "@tanstack/react-router";

import { supabase } from "@/integrations/supabase/client";

export const Route = createFileRoute("/_authenticated")({
  ssr: false,
  beforeLoad: async () => {
    const { data, error } = await supabase.auth.getUser();
    if (error || !data.user) throw redirect({ to: "/auth" });
    return { user: data.user };
  },
  component: () => <Outlet />,
});
```

6.4.2 Assessment submission

The assessment page keeps the twelve answers in a form state initialised from DEFAULTS and uses a TanStack Query mutation to call the submitAssessment server function. After a successful call the student is taken to the results page. If the prediction service is unavailable, the assessment is still saved and a warning message explains the situation.

Listing 6.18: Frontend/src/routes/_authenticated/assessment.tsx

```
const DEFAULTS: AssessmentInput = {
  age: 20,
  gender: "Female",
  country: "India",
  academic_level: "Undergraduate",
  most_used_platform: "Instagram",
  purpose_of_use: "Entertainment",
  avg_daily_usage_hours: 4,
  daily_unlocks: 60,
  study_hours: 4,
  physical_activity_hours: 1,
  sleep_hours_per_night: 7,
  stress_level: "Medium",
};

function AssessmentPage() {
  const [form, setForm] = useState<AssessmentInput>(DEFAULTS);
  const navigate = useNavigate();
  const submit = useServerFn(submitAssessment);

  const mutation = useMutation({
    mutationFn: async () => {
      console.log("ASSESSMENT_DEBUG: submitting form", form);
      return submit({ data: form });
    },
    onSuccess: (response) => {
      if (response.ok) {
        toast.success("Your assessment was saved and scored.");
      } else {
        toast.warning(
          response.message ??
            "Assessment saved, but the prediction service was unavailable."
        );
      }
      navigate({ to: "/results" });
    },
    onError: (error: Error) => {
      toast.error(error.message);
    },
  });

  function set<K extends keyof AssessmentInput>(
    key: K,
    value: AssessmentInput[K]
  ) {
    setForm((f) => ({ ...f, [key]: value }));
  }
```

6.4.3 Saving data and calling the prediction service

submitAssessment is a server function protected by the Supabase authentication middleware. It inserts the assessment with the signed-in user's id, calls the prediction service, and stores the returned score and analysis in prediction_results. Because the insert goes through the authenticated Supabase client, Row Level Security applies to every operation.

Listing 6.19: Frontend/src/lib/dashboard.functions.ts – submitAssessment

```
export const submitAssessment = createServerFn({ method: "POST" })
  .middleware([requireSupabaseAuth])
  .validator((input: unknown) => assessmentSchema.parse(input))
  .handler(async ({ data, context }) => {
    const { supabase, userId } = context;

    const { data: inserted, error } = await supabase
      .from("assessments")
      .insert({ ...data, user_id: userId })
      .select("*")
      .single();

    if (error || !inserted) {
      console.error("ASSESSMENT_SAVE_ERROR:", error);

      throw new Error(
        `Could not save your assessment: ${
          error?.message ?? "unknown error"
        }`
      );
    }

    const assessment = toAssessment(inserted as Row);

    const { predict, PredictionServiceError } = await import(
      "./prediction.server"
    );

    try {
      const prediction = await predict(assessment);
      const status = scoreStatus(prediction.score);

      const { error: resultError } = await supabase
        .from("prediction_results")
        .insert({
          user_id: userId,
          assessment_id: assessment.id,
          mental_health_score: prediction.score,
          score_category: status.label,
          feature_analysis: {
            feature_importance: prediction.feature_importance,
            model_version: prediction.model_version,
            raw: toJsonValue(prediction.raw),
          },
        });

      if (resultError) {
```

```
      throw new Error(
        `Could not save the prediction: ${resultError.message}`
      );
    }

    return {
      ok: true as const,
      assessmentId: assessment.id,
      score: prediction.score,
    };
    // ... a PredictionServiceError is caught and reported as a warning ...
```

The payload sent to the FastAPI /predict endpoint is built by toPayload(), which maps the form's field names to the names expected by the StudentInput schema. predictMentalHealth() posts the JSON, checks the HTTP status, parses the response and returns a typed result.

Listing 6.20: Frontend/src/lib/prediction.ts – payload and request

```
function toPayload(a: AssessmentInput) {
  return {
    Age: a.age,
    Gender: a.gender,
    Academic_Level: a.academic_level,
    Most_Used_Platform: a.most_used_platform,
    Purpose_Of_Use: a.purpose_of_use,
    Avg_Daily_Usage_Hours: a.avg_daily_usage_hours,
    Daily_Unlocks: a.daily_unlocks,
    Study_Hours: a.study_hours,
    Physical_Activity_Hours: a.physical_activity_hours,
    Sleep_Hours_Per_Night: a.sleep_hours_per_night,
    Stress_Level: a.stress_level,
    Country: a.country,
  };
}

export async function predictMentalHealth(
  a: AssessmentInput,
): Promise<PredictionResponse> {
  let response: Response;

  try {
    response = await fetch(PREDICTION_URL, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(toPayload(a)),
    });
  } catch {
    throw new PredictionError(
      "Could not reach the prediction server at https://students-mental-health-backend.onrender.com. Make sure your
FastAPI server is running (with CORS enabled for this app) and try again.",
    );
  }

  if (!response.ok) {
    const body = await response.text().catch(() => "");
```

```
  throw new PredictionError(
    `The prediction server returned an error (${response.status}). ${body.slice(0, 200)}`.trim(),
  );
 }
 // ... the JSON body is parsed and returned as a typed result ...
```

6.4.4 Display components

The ScoreGauge component draws the circular gauge with SVG. The progress value animates from 0 to score/10, and a short label is derived from the score (Thriving for 8 and above, Stable from 6.5, Mixed from 5, otherwise Needs Care). This display label is a friendly frontend wording and is separate from the backend's project category (Low, Moderate, Good, Excellent / Stable).

Listing 6.21: Frontend/src/components/mb/score-gauge.tsx (excerpt)

```
import { useEffect, useState } from "react";

function scoreLabel(score: number) {
  if (score >= 8) return "Thriving";
  if (score >= 6.5) return "Stable";
  if (score >= 5) return "Mixed";
  return "Needs Care";
}

export function ScoreGauge({ score, max = 10 }: { score: number; max?: number }) {
  const [progress, setProgress] = useState(0);
  const pct = Math.max(0, Math.min(1, score / max));

  useEffect(() => {
    const id = window.setTimeout(() => setProgress(pct), 120);
    return () => window.clearTimeout(id);
  }, [pct]);

  const size = 220;
  const stroke = 16;
  const r = (size - stroke) / 2;
  const c = 2 * Math.PI * r;
  const label = scoreLabel(score);

  return (
    <div className="relative flex items-center justify-center">
      <svg width={size} height={size} viewBox={`0 0 ${size} ${size}`} className="relative">
        // ... SVG circles, ticks and labels follow ...
```

6.4.5 Chat history

Chat messages are read from and written to the chat_messages table through server functions that filter by the authenticated user's id.

Listing 6.22: Frontend/src/lib/chat.functions.ts – reading chat history

```
export const getChatMessages = createServerFn({ method: "GET" })
  .middleware([requireSupabaseAuth])
  .handler(async ({ context }): Promise<ChatMessage[]> => {
    const { data, error } = await context.supabase
      .from("chat_messages")
```

```
    .select("*")
    .eq("user_id", context.userId)
    .order("created_at", { ascending: true })
    .limit(200);

  if (error) {
    throw new Error(
      `Could not load chat history: ${error.message}`
    );
  }

  return (data ?? []).map((row) => toMessage(row as Row));
});
```

6.5 Deployment

Both services are deployed on Render. The backend is a Render web service that installs the packages in requirements.txt and starts the application using the command in the Procfile. The frontend is deployed as a separate Render service built with Vite. Environment variables hold all secrets; they are never committed to Git (.env, virtual environments and node_modules are listed in .gitignore).

Listing 6.23: Backend/Procfile

```
web: uvicorn api.main:app --host 0.0.0.0 --port $PORT
```

Table 6.1: Environment variables

Service	Variable	Purpose
Backend	GEMINI_API_KEY	API key for the Google Gemini Wellness Assistant
Backend	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY	Connection to the Supabase project (server side only)
Backend	MODEL_URL	Optional download location of final_random_forest.pkl
Backend	PORT	Provided by Render; used by Uvicorn
Frontend	VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY	Supabase project URL and public key for the browser client
Frontend	PREDICTION_API_URL	Base URL of the deployed FastAPI backend

Running the project locally

1. Clone the repository: git clone https://github.com/rutuu0228/Students_Mental_Health_Score_Prediction_sys.git
2. Backend: cd Backend, create and activate a virtual environment, run pip install -r requirements.txt, set the environment variables and start the server with uvicorn api.main:app --reload.
3. Open http://127.0.0.1:8000/docs to try the /predict endpoint in Swagger UI.
4. Frontend: cd Frontend, run npm install and npm run dev, and set the Supabase variables and PREDICTION_API_URL (for local work http://127.0.0.1:8000).

Table 6.2: Live deployment links

Service	Link
VRITTACARE frontend	https://students-mental-health-frontend.onrender.com

Service	Link
FastAPI backend	https://students-mental-health-backend-1.onrender.com
API documentation	https://students-mental-health-backend-1.onrender.com/docs
Health check	https://students-mental-health-backend-1.onrender.com/health
GitHub repository	https://github.com/rutuu0228/Students_Mental_Health_Score_Prediction_sys

Render services on free or sleeping plans may need a short warm-up period after inactivity.

6.6 Testing

The prediction pipeline was tested with sample student profiles that represent different routines. In every case the score, the category and the grouped observations were consistent with the patterns found in the data analysis: healthier routines gave higher scores and fewer areas needing attention.

Case	Input summary	Score	Category	Areas needing attention
T1	Usage 2.5 h, 40 unlocks, study 5 h, activity 2.5 h, sleep 8 h, stress Low	7.59	Good	None
T2	Usage 4.5 h, 60 unlocks, study 3 h, activity 1 h, sleep 7 h, stress Medium	6.84	Moderate	Physical activity
T3	Usage 7.2 h, 95 unlocks, study 6 h, activity 1 h, sleep 5 h, stress High	5.94	Moderate	Screen usage, physical activity, sleep, stress
T4	Usage 8 h, 220 unlocks, study 1 h, activity 0.2 h, sleep 4.5 h, stress Very High	5.00	Moderate	All six areas

The tuned model reached MAE 0.2670, RMSE 0.3586 and R^2 0.9280 on 1,000 unseen records. Inputs outside the ranges seen in the training data (for example fewer than 62 phone unlocks per day) are handled by the Random Forest using the nearest learned pattern, so such results should be read with care.

Table 6.3: Implementation summary

Component	Technology	Main files
Data and ML workflow	Python, pandas, scikit-learn, Jupyter	Notebooks 01 to 06, Data folder, Models/preprocessor.pkl
Prediction service	FastAPI, Pydantic, joblib	api/main.py, api/schemas.py, api/routes/prediction.py, predictor.py
Feature analysis	Python rules and what-if prediction	feature_analysis.py
Wellness Assistant	Google Gemini API	gemini_chatbot.py, api/routes/chatbot.py, chat.functions.ts
Web application	React, TypeScript, TanStack Start, Tailwind CSS	src/routes, src/components/mb, src/lib
Authentication and data	Supabase Auth and PostgreSQL	src/integrations/supabase, use-auth.ts, dashboard.functions.ts
Deployment	Render, Git and GitHub	Procfile, requirements.txt, package.json

7. FUTURE SCOPE OF PROJECT

VRITTACARE can be extended in many useful directions in the future:

1. Larger and more diverse data: retraining with a larger, carefully collected dataset (with consent) and aligning every option in the form with the categories seen in training, so predictions become more reliable for different groups of students.
2. Progress tracking: weekly check-ins with trend charts so that a student can see how habits and scores change over time.
3. Explainable predictions: adding techniques such as SHAP to explain each individual prediction in more detail.
4. Mobile application: a mobile app or installable web app with reminders and notifications.
5. Multilingual support: Marathi and Hindi support in the interface and the Wellness Assistant.
6. Wearable integration: reading sleep and activity data from wearable devices to reduce manual entry.
7. Counsellor and college dashboard: anonymised, consent-based summary insights for college counsellors.
8. Safety features: detecting signs of serious distress in the chat and showing trusted helpline and support information.
9. Guided activities: short breathing, mindfulness and habit-building exercises linked to the observations.
10. Production hardening: model versioning and monitoring, periodic retraining, API authentication, rate limiting and restricting CORS to the frontend address.

8. CONCLUSION

The project VRITTACARE successfully demonstrates how data science, a REST API and a modern web application can be combined into one complete system for student well-being. Starting from a dataset of 5,000 student records, the data was cleaned, explored and prepared, and four regression algorithms were compared. The tuned Random Forest model was selected as the final model and reached an R^2 score of 0.9280 with a Mean Absolute Error of 0.2670 on unseen data.

The model was packaged with its preprocessing pipeline, served through a validated FastAPI service and connected to a React web application with secure sign-in and user-scoped data storage in Supabase. The application turns the model output into a score gauge, a project-defined category, grouped observations, practical suggestions, a 3D wellness profile and a supportive Wellness Assistant, so that the result is understandable and not just a number.

The project also gave the team practical experience in the complete data-science life cycle: data cleaning, exploratory analysis, feature engineering, model selection and tuning, API design, frontend development, database design, deployment and documentation.

VRITTACARE has limits that must be understood. The score is a model-generated estimate and not a diagnosis. The category labels and feature rules are project-defined guidance and are not clinical thresholds. Patterns learned from historical data do not prove that one habit causes a mental-health outcome, and the results depend on the coverage and quality of the training data. The application is meant for learning and self-reflection, not for clinical decisions or emergency support. Within these limits, VRITTACARE shows how technology can encourage students to look at their daily habits and take small, thoughtful steps towards better well-being.

9. BIBLIOGRAPHY

9.1 Books

1. Aurélien Géron, “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow”, 3rd edition, O’Reilly Media, 2022.
2. Wes McKinney, “Python for Data Analysis”, 3rd edition, O’Reilly Media, 2022.
3. Sebastian Raschka, and Vahid Mirjalili, “Python Machine Learning”, 3rd edition, Packt Publishing, 2019.
4. Roger S. Pressman, and Bruce R. Maxim, “Software Engineering: A Practitioner’s Approach”, 9th edition, McGraw-Hill Education, 2019.
5. Jeffrey L. Whitten, and Lonnie D. Bentley, “Systems Analysis and Design Methods”, 7th edition, McGraw-Hill/Irwin, 2007.

9.2 Websites

1. https://github.com/rutuu0228/Students_Mental_Health_Score_Prediction_sys
2. https://students-mental-health-frontend.onrender.com/
3. https://students-mental-health-backend-1.onrender.com/docs
4. https://fastapi.tiangolo.com/
5. https://scikit-learn.org/stable/
6. https://pandas.pydata.org/docs/
7. https://react.dev/
8. https://supabase.com/docs
9. https://render.com/docs
10. https://ai.google.dev/gemini-api/docs